

Rapport de projet

Déploiement d'un LLM open-source avec Docker et Kubernetes

Cheikh Oumar DIALLO

4 septembre 2026

1- Objectifs du projet

Ce projet consiste à déployer une stack applicative composée d'Ollama, moteur d'inférence pour modèles de langage, et d'Open WebUI, une interface web de type chat. Le déploiement est d'abord réalisé avec Docker Compose, puis migré vers Kubernetes à l'aide de Minikube, en appliquant les bonnes pratiques d'orchestration et de gestion des conteneurs.

2- Rôle de chaque ressource Kubernetes

On va essayer de présenter chaque ressource pour une meilleure compréhension du projet dans sa globalité avec le rôle pour chaque ressource. Tous les fichiers sont dans le dossier **k8s/**

Ressources	Fichier	Rôle
Namespace	00-namespace.yaml	Isole toutes les ressources du projet du reste du cluster, nommé ici par “projet-devops” évite les collisions de noms avec d'autres projets partageant le même Minikube.
ConfigMap	01-configmap.yaml	Centralise la configuration non-sensible, tout ce qui est ports, URL interne d'Ollama, injectée dans les conteneurs via envFrom, sans la coder en dur dans les manifests de déploiement.
PersistentVolume / PersistentVolumeClaim	02-ollama-pv.yaml, 03-ollama-pvc.yaml, 04-openwebui-pv.yaml, 05-openwebui-pvc.yaml	Fournissent un stockage qui survit au cycle de vie d'un pod, voir section 4 pour plus d'infos.
Deployment Ollama	06-ollama-deployment.yaml	Gère le pod exécutant le moteur Ollama : image, ressources CPU/RAM, probes de santé, montage du volume de modèles.
Deployment Open WebUI	08-openwebui-deployment.yaml	Gère le pod de l'interface web, connecté à Ollama via le réseau interne du cluster.
Service Ollama et Open WebUI	07-ollama-service.yaml, 09-openwebui-service.yaml	Exposent chaque Deployment sous un nom DNS stable et une IP virtuelle interne (ClusterIP), indépendants du pod réellement en cours (qui peut être recréé avec une IP différente)

Ingress	10-ingress.yaml	Point d'entrée HTTP unique routant vers Open WebUI avec "/" et Ollama avec "/ollama". C'est une alternative au port-forward pour un accès externe plus proche d'un usage production.
---------	-----------------	--

3- Communication entre les services

Avec Docker Compose, les conteneurs communiquent directement sur un réseau **bridge** partagé en utilisant leur nom de conteneur, par exemple `http://ollama:11434`. Avec Kubernetes, cette logique est différente : le nom du conteneur n'est pas utilisé comme point d'accès réseau. C'est le **Service Kubernetes** qui fournit une adresse réseau stable aux applications.

Concrètement, lorsque le pod `open-webui` souhaite communiquer avec Ollama, il utilise le nom DNS `ollama-service`. **CoreDNS**, le service DNS interne du cluster, résout ce nom vers l'adresse IP virtuelle du Service.

Le Service a ensuite pour rôle d'acheminer le trafic vers le pod `ollama` correspondant aux sélecteurs configurés. Le pod peut être supprimé puis recréé avec une nouvelle adresse IP sans que `open-webui` ait besoin de connaître ce changement. Le Service constitue donc une abstraction stable entre le client et les pods.

Le Service peut également exposer un port différent de celui utilisé par le conteneur. Dans notre cas, le Service expose le port `80` et redirige les requêtes vers le port `11434` du conteneur grâce à `targetPort`. Cette séparation permet de modifier l'implémentation interne sans modifier l'adresse utilisée par les clients.

Enfin, une **NetworkPolicy** peut renforcer cette architecture en contrôlant les communications autorisées. Elle peut notamment limiter l'accès à Ollama au seul pod `open-webui` sur le port `11434`, empêchant ainsi les autres pods non autorisés du namespace d'accéder directement au service.

4- Pourquoi des PersistentVolumeClaims

Un Pod Kubernetes est éphémère. Lorsqu'il est recréé, les données stockées dans son filesystem local peuvent être perdues.

Cette problématique est importante pour **Ollama**, qui stocke les modèles téléchargés localement. Sans stockage persistant, ces modèles devraient être téléchargés à nouveau après chaque recréation du Pod.

Pour **Open WebUI**, la persistance concerne notamment la base SQLite et les données stockées dans `/app/backend/data`, qui contiennent les utilisateurs et l'historique des conversations.

La **PersistentVolumeClaim (PVC)** permet de conserver ces données indépendamment du cycle de vie du Pod. Elle se lie à un **PersistentVolume (PV)** qui fournit l'espace de stockage. Dans Minikube, les PV utilisent ici un stockage `hostPath`.

Lorsqu'un Pod est recréé, il remonte la même PVC et retrouve ainsi les données précédemment stockées.

Dans ce projet, le binding est **statique**, avec `storageClassName: manual` et un `volumeName` explicite. Cela permet de contrôler précisément l'association entre chaque PVC et son PV.

5- Rôle des probes startup, readiness, liveness

Les trois probes Kubernetes vérifient des aspects différents de l'état d'un conteneur.

La **Startup Probe** vérifie que l'application a terminé son démarrage. Elle est particulièrement utile pour Open WebUI, dont l'initialisation peut être longue à cause du chargement du modèle d'embedding.

La **Readiness Probe** vérifie si le conteneur peut recevoir du trafic. En cas d'échec, le Pod est temporairement retiré des endpoints du Service, sans être redémarré.

La **Liveness Probe** vérifie si l'application fonctionne toujours correctement. En cas d'échecs persistants, Kubernetes redémarre le conteneur. Elle permet notamment de récupérer automatiquement d'un processus bloqué.

Lors des tests, une autre situation a été observée avec `llama-server`, terminé avec le statut **OOMKilled** en raison d'un manque de mémoire. Les probes permettent alors de constater l'indisponibilité, mais le diagnostic nécessite d'examiner les logs et l'état du Pod pour identifier la cause réelle.

6- Docker compose vs Kubernetes

Aspect	Docker Compose	Kubernetes
Portée	Principalement un seul hôte	Cluster d'un ou plusieurs nœuds
Découverte de service	Noms des services sur un réseau Docker	Services + DNS interne (CoreDNS)



Auto-guérison	Redémarrage avec restart	Controllers + probes
Stockage	Volumes Docker	PV / PVC
Scalabilité	Scaling manuel	HPA pour le scaling automatique
Configuration	.env et variables d'environnement	ConfigMap / Secret
Réseau	Réseaux Docker	NetworkPolicy pour un contrôle plus fin

En résumé, Docker Compose est adapté au développement local et aux environnements simples, tandis que Kubernetes apporte des mécanismes avancés d'orchestration, de résilience, de scaling et de contrôle réseau, particulièrement utiles pour des environnements de production.

7. Fonctionnalités bonus implémentées

Les fonctionnalités suivantes ont également été implémentées et validées : **NetworkPolicy**, **PodDisruptionBudget**, **HorizontalPodAutoscaler**, **GitHub Actions**, **Kustomize** pour **development** et **production**, **Helm**, ainsi qu'une stack de **monitoring Prometheus + Grafana**.

Conclusion

L'ensemble des fonctionnalités a été testé sur container Minikube disposant de **7,6 Go de RAM**, également utilisée par d'autres projets. Les limites de ressources définies dans les manifests sont donc principalement liées aux capacités de la machine de test et non à des contraintes propres à Kubernetes. Ce fut un projet riche en connaissances surtout sur la partie Kubernetes.

